# Apache Flink Kickstarter II **[UNDER CONSTRUCTION]**

> ***On GitHub, Watch → Custom → Releases is the most useful setting if you want to stay updated on when something important is released for this project.***

**Apache Flink Kickstarter II** is the 2026 evolution of my original Kickstarter project — rebuilt to showcase the cutting edge of the [Confluent distribution of Open Software Source (OSS)] **Apache Flink 2.1.x**.

Designed as a hands-on, production-minded accelerator, it brings Flink to life *locally* on **Confluent Platform on Minikube**, while drawing direct comparisons to **Confluent Cloud for Apache Flink** — so you can clearly see what's possible across environments.

Every **example** is delivered end-to-end — from schema design to fully operational streaming pipelines — with implementations in **both Java and Python (when possible)** where it matters, bridging real-world developer workflows with modern streaming architecture.

---

**Table of Contents**


- **1.0 Prerequisites**
  - **1.1 Confluent Platform on Minikube — Production-Like Streaming, Running Locally**
  - **1.2 Confluent Cloud**
- **2.0 The Examples**
  - **2.1 Apache Flink User-Defined Functions (UDF)**
    - **2.1.1 Process Table Functions (PTF)**
      - **2.1.1.1 Limitation(s)**
    - **2.1.2 Scalars**
- **3.0 Debugging the Examples**
  - **3.1 Apache Flink UDF Debugging with Java Debug Wire Protocol (JDWP)**
    - **3.1.1 Process Table Functions (PTF)**
      - **3.1.1.1 Debugging the `row-driven` PTF using set semantics (`UserEventEnricher`)**
      - **3.1.1.2 Debugging the `row-driven` PTF using row semantics (`OrderLineExpander`)**
      - **3.1.1.3 Debugging the `timer-driven` PTFs (`SessionTimeoutDetector`, `AbandonedCartDetector`, `PerEventFollowUp`, and `SlaMonitor`)**
    - **3.1.2 Scalars**
      - **3.1.2.1 Debugging the Celsius to Fahrenheit Scalar UDF (`CelsiusToFahrenheit`)**
      - **3.1.2.2 Debugging the Fahrenheit to Celsius Scalar UDF (`FahrenheitToCelsius`)**
- **4.0 Resources**
  - **4.1 Confluent for Kubernetes (CfK)**

- **4.2 Confluent Platform for Apache Flink**
- **4.3 Confluent Cloud for Apache Flink (CCAF)**


---

# 1.0 Prerequisites

First, clone the repo to your local machine using the GitHub CLI:

```
gh repo clone j3-signalroom/apache_flink-kickstarter-ii
```

Change to the repo directory:

```
cd /path/to/apache_flink-kickstarter-ii
```

Then decide where you want to run the examples:

## 1.1 Confluent Platform on Minikube — Production-Like Streaming, Running Locally

To ***run***, ***test***, and ***debug*** Apache Flink like a production engineer, this project provides a full Confluent Platform stack running locally on Minikube — no cloud required.

You get a production-like environment on your machine, with all the components you'd expect in a real deployment:

- **Confluent Platform** (KRaft mode) via Confluent for Kubernetes (CFK)
- **Apache Flink 2.1.1** via the Confluent Flink Kubernetes Operator 1.130
- **Confluent Manager for Apache Flink (CMF) 2.1** for Flink environment management

To run this project, you'll need **macOS (with Homebrew)** or **Linux (with apt-get)**. The full stack — **Minikube + Confluent Platform + Flink + CMF** — is resource-intensive and designed to mirror a production environment. Therefore, the following defaults are recommended:

| Resource | Default |
|---|---|
| CPUs | 6 |
| Memory | 20 GB |
| Disk | 50 GB |

> These settings ensure stable performance across all components. You can tune them as needed, but lower resource levels may cause pod restarts or degraded performance.

👉 If your machine meets all the requirements, click **here**. Otherwise, click **here** to set up a *Remote Server Setup (SSH Tunneling)* on a provider like Vultr VPS.

🧠 ***Build locally. Debug with confidence. Deploy to production-ready environments.***

---

### 1.2 Confluent Cloud

In addition to running the examples **locally**, you can also run them in Confluent Cloud. The examples are designed to mirror the local Confluent Platform setup as closely as possible, so you can easily compare and contrast the two environments. Or, better yet, run the same example in both environments to confirm that they behave the same way.

Before you begin, ensure you have access to the following cloud accounts:

- **Confluent Cloud Account** — for Kafka, Schema Registry, and Flink resources
- **Terraform Cloud Account** — for automated infrastructure provisioning

Make sure the following tools are installed on your local machine:

- **Java JDK 21** — for building Java based Flink UDFs
- **Gradle 9.4.1 or higher** — for building Java based Flink UDFs
- **Python 3.11** — for building Python based Flink UDFs
- **uv** — for building Python based Flink UDFs
- **Terraform CLI version 1.13.0 or higher** — for deploying infrastructure to Confluent Cloud

---

## 2.0 The Examples

Once you've set up **Confluent Platform on Minikube** or have your **Confluent Cloud** account with the prerequisites in place, you're ready to try the examples:

### 2.1 Apache Flink User-Defined Functions (UDF)

#### 2.1.1 Process Table Functions (PTF)

▶ ***What are PTFs?***
▶ ***Why use PTFs?***
▶ ***When do you use PTFs?***
▶ ***Where do you use PTFs?***
▼ ***How are examples of PTFs put into practice?***

| Type | Purpose | Confluent Platform on Minikube | Confluent Cloud |
|---|---|---|---|
| PTF UDF-type (`row-driven`)<br><br>Java<br>Python | Walks through both **local** and cloud environments *building*, *deploying*, and *testing* two `row-driven` **PTF UDFs** (no timers) bundled in one JAR that illustrate both `ArgumentTrait` modes: **User Event Enricher** (`SET_SEMANTIC_TABLE` — enriches Kafka user events with per-user session tracking using keyed state) and **Order Line Expander** (`ROW_SEMANTIC_TABLE` — stateless one-to-many expansion of an order row into individual line items). | `CP Deploy` | `CC Deploy` |

| Type | Purpose | Confluent Platform on Minikube | Confluent Cloud |
|---|---|---|---|
| PTF UDF-type (`timer-driven`)<br><br>Java<br>Python | Walks through both **local** and cloud environments *building*, *deploying*, and *testing* four **`timer-driven` PTF UDFs** bundled in one JAR: **Session Timeout Detector** (named timers using the inactivity pattern), **Abandoned Cart Detector** (named timers using the inactivity pattern for e-commerce), **Per-Event Follow-Up** (unnamed timers using the scheduling pattern), and **SLA Monitor** (unnamed timers using the scheduling pattern). | `CP Deploy` | `CC Deploy` |

⏳ **PTF Rules of Thumb: The Hourglass Pattern**

#### 2.1.1.1 Limitation(s)

- Why `@StateHint` POJO with `Map` or `List` Are Sensitive to "Extremely Large State"

### 2.1.2 Scalars

▶ ***What are Scalars?***
▶ ***Why use Scalars?***
▶ ***When do you use Scalars?***
▶ ***Where do you use Scalars?***
▼ ***How are examples of Scalars put into practice?***

| Type | Purpose | Confluent Platform on Minikube | Confluent Cloud |
|---|---|---|---|
| Scalar UDF-type<br><br>Java<br>Python | Walks through both **local** and cloud environments *building*, *deploying*, and *testing* **eight Scalar UDFs** bundled in a single JAR that illustrate the simplest and oldest category of Flink user-defined function — a pure, row-at-a-time function callable from SQL. The eight UDFs span three themes. The **unit-conversion primer** (the "Hello, UDF" pair) is **`CelsiusToFahrenheit`** and **`FahrenheitToCelsius`** — plain `DOUBLE → DOUBLE` arithmetic, the smallest possible shape of a scalar UDF. The **PII & data-governance** quartet is **`PseudonymizePii`** (keyed HMAC-SHA256 pseudonym with a namespace argument for domain separation — irreversibly hides PII from analysts while keeping joins and `GROUP BY` stable), **`ResolveIdentity`** (un-keyed SHA-256 of a per-type normalized identity signal — a portable canonical join key across systems that don't share a secret), **`TokenizePan`** (tokenizes a PAN into a `{BIN}{HMAC-hex middle}{last-4}` shape, taking the Flink pipeline out of PCI DSS scope while | `CP Deploy` | `CC Deploy` |

| Type | Purpose | Confluent Platform on Minikube | Confluent Cloud |
|---|---|---|---|
| | preserving first-6 and last-4 for fraud analytics and reconciliation), and **`MaskPiiRegex`** (free-form-text PII masking using Microsoft Presidio's regex recognizers, with Java and Python implementations that produce byte-identical output for the same input — the Python side calls Presidio directly, the Java side ports its patterns and validation logic verbatim). The **pipeline-reliability** pair is **`DedupKey`** (length-prefix-encoded SHA-256 over an `ARRAY<STRING>` of business fields — a stable idempotency key that defangs at-least-once duplicates via `ROW_NUMBER() OVER (PARTITION BY dedup_key …)`) and **`ConsistentBucket`** (Kafka-compatible 32-bit `murmur2`, the same hash Kafka's default partitioner uses — so Flink-computed buckets match Kafka-producer partitions, database shards, and cache slots byte-for-byte). | | |

---

# 3.0 Debugging the Examples

## 3.1 Apache Flink UDF Debugging with Java Debug Wire Protocol (JDWP)

You can attach your IDE's debugger (VS Code or IntelliJ IDEA) to a running Flink TaskManager and *hit breakpoints inside your UDF code* — even though it's executing on a remote Java Virtual Machine (JVM) inside Kubernetes. The `FlinkDeployment` Custom Resource (CR) already has **Java Debug Wire Protocol (JDWP)** enabled, and debug configurations are pre-wired for both VS Code and IntelliJ IDEA.

**Prerequisites:** The Confluent Platform and Flink stack must be running (`make cp-up && make flink-up`), and your UDF must be deployed.

### 3.1.1 Process Table Functions (PTF)

#### 3.1.1.1 Debugging the `row-driven` PTF using set semantics (`UserEventEnricher`)

> For the full deep-dive, see Remote Debugging `row-driven` Flink PTF UDFs. The same guide also covers debugging `OrderLineExpander` (§1.3).

Deploy first: `make deploy-cp-ptf-udf-row-driven`, and then:

- ▶ 1. Set a breakpoint
- ▶ 2. Attach the debugger
- ▶ 3. Send a test event
- ▶ 4. Debug

#### 3.1.1.2 Debugging the `row-driven` PTF using row semantics (`OrderLineExpander`)

> For the full deep-dive, see Remote Debugging `row-driven` Flink PTF UDFs (§1.3).

> The `OrderLineExpander` ships in the **same uber JAR** as `UserEventEnricher`, so the same `make deploy-cp-ptf-udf-row-driven` command and the same **"Attach to Flink TaskManager (`row-driven`)"** debug configuration are used. The deploy script registers both PTFs as separate Flink SQL functions and starts an `INSERT INTO orders_expanded SELECT ... FROM TABLE(order_line_expander(input => TABLE orders))` pipeline alongside the user-event enrichment job.

Deploy first: `make deploy-cp-ptf-udf-row-driven`, and then:

- ▶ 1. Set a breakpoint
- ▶ 2. Attach the debugger
- ▶ 3. Send a test order
- ▶ 4. Debug

#### 3.1.1.3 Debugging the `timer-driven` PTFs (`SessionTimeoutDetector`, `AbandonedCartDetector`, `PerEventFollowUp`, and `SlaMonitor`)

> For the full deep-dive, see Remote Debugging `timer-driven` Flink PTF UDFs.

Deploy first: `make deploy-cp-ptf-udf-timer-driven`, and then:

- ▶ 1. Set a breakpoint
- ▶ 2. Attach the debugger
- ▶ 3. Send a test event
- ▶ 4. Debug

### 3.1.2 Scalars

#### 3.1.2.1 Debugging the Celsius to Fahrenheit Scalar UDF (`CelsiusToFahrenheit`)

> For the full deep-dive, see Remote Debugging Flink Scalar UDFs (§1.2).
>
> The `CelsiusToFahrenheit` ships in the **same uber JAR** as `FahrenheitToCelsius`, so the same `make deploy-cc-scalar-udf` command and the same **"Attach to Flink TaskManager (scalar udf)"** debug configuration are used. The deploy script registers both Scalars as separate Flink SQL functions and starts an `INSERT INTO celsius_to_fahrenheit (sensor_id, celsius_temperature, fahrenheit_temperature) SELECT sensor_id, celsius_temperature, celsius_to_fahrenheit(celsius_temperature) FROM celsius_reading;` pipeline alongside the scalar udf examples job.

Deploy first: `make deploy-cc-scalar-udf`, and then:

- ▶ 1. Set a breakpoint
- ▶ 2. Attach the debugger
- ▶ 3. Send a test order
- ▶ 4. Debug

#### 3.1.2.2 Debugging the Fahrenheit to Celsius Scalar UDF (`FahrenheitToCelsius`)

> For the full deep-dive, see Remote Debugging Flink Scalar UDFs (§1.3).

> The `FahrenheitToCelsius` ships in the **same uber JAR** as `CelsiusToFahrenheit`, so the same `make deploy-cc-scalar-udf` command and the same **"Attach to Flink TaskManager (scalar udf)"** debug configuration are used. The deploy script registers both Scalars as separate Flink SQL functions and starts an `INSERT INTO fahrenheit_to_celsius (sensor_id, fahrenheit_temperature, celsius_temperature) SELECT sensor_id, fahrenheit_temperature, fahrenheit_to_celsius(fahrenheit_temperature) FROM fahrenheit_reading;` pipeline alongside the scalar udf examples job.

Deploy first: `make deploy-cc-scalar-udf`, and then:

▶ 1. Set a breakpoint
▶ 2. Attach the debugger
▶ 3. Send a test order
▶ 4. Debug

---

# 4.0 Resources

## 4.1 Confluent for Kubernetes (CfK)

- Manage Confluent Platform with Confluent for Kubernetes
- Minikube

## 4.2 Confluent Platform for Apache Flink

- Stream Processing with Confluent Platform for Apache Flink
- Architecture and Features of Confluent Platform for Apache Flink
- Get Started with Confluent Platform for Apache Flink

## 4.3 Confluent Cloud for Apache Flink (CCAF)

- Stream Processing with Confluent Cloud for Apache Flink
- Get Started with Confluent Cloud for Apache Flink
- User-defined Functions in Confluent Cloud for Apache Flink